GAP : Chromium-Based Vulnerability

A Fileless Extension Persistence Technique in Chromium-based Browsers

Corentin MAHIEU

https://fir3n0x.github.io

June 2026

Abstract

Browser extensions represent an often overlooked yet meaningful attack surface in modern enterprise environments. While endpoint detection and response (EDR) solutions focus on filesystem and process activity, the internal architecture of Chromium-based browsers creates a blind spot that can be exploited for persistent, fileless code execution after an initial access. This paper presents GAP (Ghost Anchor Persistence), a persistence technique I discovered during my research, affecting Chromium-based browsers like Edge, Chrome, Vivaldi or Brave. GAP exploits an architectural decoupling between the browser's Secure Preferences file, its Service Worker Registration database (LevelDB), and its ScriptCache. By injecting a malicious extension whose service worker is cached in ScriptCache, then replacing the malicious folder with a benign one sharing the same extension ID, an attacker achieves persistent arbitrary JavaScript execution within the browser context, with no malicious artifact remaining on the Windows filesystem. The technique survives browser restarts and updates, bypasses filesystem-based AV/EDR scanning, and leaves only a benign extension folder as forensic evidence. We demonstrate GAP against Microsoft Edge for Business on Windows 10 and Windows 11 in both personal and Active Directory environments, provide a full proof-of-concept, and discuss detection strategies and recommended mitigations. A responsible disclosure was submitted to the Microsoft Security Response Center (MSRC Case 112111) prior to publication.

Contents

1 Introduction

Browser extensions are an under-estimated threat in the cybersecurity ecosystem. Indeed, they are legitimate by design and natively trusted by the browser which represent an ideal implant vector for red team operators seeking persistence within an enterprise environment. For instance, malicious browser extensions can exfiltrate sensitive information, retrieve credentials, analyze user activity, inject code in DOM page as a keylogger, and in a certain configuration, even jailbreak the browser sandbox to execute arbitrary system commands. While, extension-based attacks are not new, the internal mechanics of Chromium-based browsers introduce an architectural quirk that has to our knowledge, not been documented yet. A structural decoupling between the browser's configuration layer, its Service Worker registration database, and its script cache can be leveraged to create a stealthy, cache-based extension persistence.

This paper presents GAP (Ghost Anchor Persistence), a persistence technique affecting Microsoft Edge and, by extension, Chromium-based browsers. GAP allows an attacker with an initial foothold on a Windows machine to achieve persistent, arbitrary JavaScript execution within the browser, with no malicious artifact remaining on the filesystem. The only evidence left on disk is a benign extension folder containing empty scripts. The malicious service worker lives exclusively in Edge's internal ScriptCache, currently invisible to most filesystem-based AV/EDR tools and forensic analysis.

Contributions. This paper makes the following contributions:

- We document GAP, a novel fileless persistence technique exploiting an architectural decoupling in Chromium-based browsers.
- We present `stomp` [1], a toolchain enabling extension injection and GPO bypass in Microsoft Edge for Business.
- We provide forensic evidence of the decoupling through ScriptCache and Service Worker Database analysis.
- We discuss detection strategies and recommended mitigations for both Microsoft and blue team operators.

2 Background

In order to truly understand the vulnerability, we must cover some technical aspects related to browsers and extensions.

2.1 Chromium Extension Architecture

Modern Chromium-based browser (Edge, Chrome, Brave, Vivaldi, etc.) implement extensions as sandboxed web applications governed by a `manifest.json` file. This file, included in each extension, declares their identity, permissions, and entry points. Extensions operate across three distincts execution context. A *background service worker* running persistently in a dedicated thread, *content scripts* injected into web pages, and *popup scripts* attached to the browser UI which is less important in our context. From an attacker's perspective, the background service worker is the most valuable target: it runs continuously (actually, periodically), has access to privileged browser APIs (`cookies`, `webRequest`, `scripting`), and is not directly observable by the user. As it is running

in an extension's background, that's the most suitable place to store and run arbitrary scripts in an attacker point of view.

2.2 Secure Preferences

Chromium-based browsers persist their configuration, including installed extensions, their filesystem paths, and associated metadata, in a JSON file named `Secure Preferences`, located at:

```
%LOCALAPPDATA%\Microsoft\Edge\User Data\Secure Preferences
```

To prevent unauthorized modification, this file is signed with an HMAC computed over its content at each browser launch. Any tampering without recalculating a valid HMAC causes the browser to reject the modifications or reinitialize the file. Prior work by Synacktiv [2] or other papers like *HMAC and "Secure Preferences": Revisiting Chromium-based Browsers Security* [3] demonstrated that this HMAC can be recomputed offline given the target machine's SID, enabling arbitrary extension injection without user interaction.

2.3 Service Worker Registration

When an extension declaring a background service worker is first launched, Chromium registers it in an internal LevelDB database located in the filesystem at:

```
Service Worker\Database\
```

This *Registration* is a persistent record mapping an extension scope (`chrome-extension://<ID>/`) to its service worker script URL and an internal `version_id`. A typical entry takes the following form:

```
key   : chrome-extension://<ID>/
value : {
  script_url: "chrome-extension://<ID>/background.js",
  version_id: N,
  script_cache_key: X
}
```

Critically, this database is **architecturally independent** from the extension's folder on the filesystem. At each browser restart, Edge restores all Registrations from this database regardless of the current state of the corresponding extension folders.

2.4 ScriptCache

Chromium caches compiled service worker scripts in a dedicated directory:

```
Service Worker\ScriptCache\
```

Each entry consists of a pair of files indexed by a hash derived from the script URL: a `_0` file containing the compiled JavaScript, and a `_1` file storing associated metadata, as shown in Figure 1. The ScriptCache is invalidated only under three conditions: (1) the script is served over HTTP with stale cache-control headers, (2) an explicit

`unregister()` call is made, or (3) the extension is updated through the official browser update mechanism. Notably, **replacing the extension folder on the filesystem does not trigger ScriptCache invalidation**.

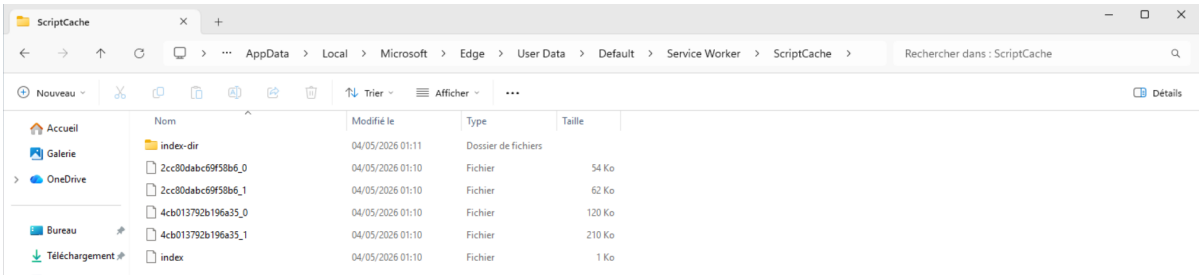

Figure 1: ScriptCache directory.

2.5 Related Work

Extension-based attacks have been documented across multiple threat contexts. Malicious extensions distributed through official stores have been used for credential theft [4], ad injection [5], and C2 communication [6]. On the injection side, Synacktiv's *extloader* [2] established that arbitrary extensions could be force-installed in Chromium browsers by manipulating the Secure Preferences file, even in environments governed by Group Policy Objects (GPO). Our work builds on this primitive and identifies a deeper architectural flaw that enables fileless persistence, a property not achieved by prior injection techniques.

3 Threat Model

The vulnerability presented in this paper is a post exploitation technique which requires some prerequisites.

3.1 Attacker Assumptions

GAP requires an initial foothold on the target machine. Specifically, the attacker must be able to execute arbitrary code, like a *.bat* script to set up the vulnerability, in the context of the target user, a condition commonly satisfied in post-exploitation scenarios via phishing, supply chain compromise, or lateral movement. No administrative privileges are required: standard user rights are sufficient to read and overwrite the `Secure Preferences` file and copy folders to `%LOCALAPPDATA%` or wherever the attacker wants to. The attacker is assumed to have prior knowledge of the target's SID value, retrievable via `whoami /user`.

3.2 Objectives

The attacker's goals are twofold:

- **Persistence** - maintain arbitrary JavaScript execution within the browser across reboots and browser updates, without relying on any malicious artifact on the filesystem.

- **Evasion** - avoid detection by filesystem-based AV/EDR solutions and forensic analysis tools by leaving only a benign extension folder on disk.

3.3 Target Environments

GAP has been validated against Microsoft Edge for Business in the following configurations:

OS	Configuration
Windows 11	Active Directory + GPO
Windows 11	Personal account + GPO
Windows 11	Personal account
Windows 10	Personal account

Table 1: Tested environments

Based on Windows 11 environment with a Personal account, GAP has also been validated for the following Chrome, Brave and Vivaldi Chromium-based browsers.
GPO environments are of particular interest: organizations commonly deploy allowlists restricting which extensions may run in the browser. GAP's injection primitive addresses this constraint through extension ID spoofing, as detailed in Section 4.

3.4 Scope and Limitations

GAP affects background service workers exclusively. Content scripts, which are loaded directly from the extension folder at injection time, do not benefit from ScriptCache persistence and are therefore not covered by this technique. This limitation is addressed in Section 6, where we demonstrate that dynamic content script injection via `scripting.executeScript()` achieves equivalent capabilities without any additional filesystem artifacts.

4 Extension Injection in Edge for Business

In most environments, installing browser extensions is prohibited. By subtly tampering with `Secure Preferences` file, we can bypass restrictions usually enforced by GPOs policies.

4.1 Secure Preferences Manipulation

The foundation of our injection primitive relies on offline manipulation of the `Secure Preferences` file. Given the target's current `Secure Preferences` and SID value, an attacker can generate a valid replacement file, including a correctly computed HMAC, that registers an arbitrary extension. This approach, first documented by Synacktiv [2], does not require browser interaction during generation and produces a file indistinguishable from a legitimately modified one.

I developed `stomp`, a toolchain extending this primitive with GPO bypass capabilities. Given an extension directory and target parameters, `stomp` produces a deployment archive containing:

- `extension/` - the extension folder to deploy

- `inject.bat` - automated deployment script
- `preferences/` - browser-specific generated `Secure Preferences` - new generated file to inject
- `SecurePreferencesClean` — backup of the original file

```
python3 stomp.py EXTENSION_A/ \
  --prefs-file SecurePreferences \
  --sid "S-1-5-21-XXX-XXX-XXX-XXX" \
  --browser "edge" \
  --target-dir "%LOCALAPPDATA%"
```

Once transferred to the target machine, the attacker executes `inject.bat`, which kills any running Edge instance, restores a clean `Secure Preferences` to avoid state collisions, copies the extension folder to `%LOCALAPPDATA%`, and overwrites `Secure Preferences` with the generated file. On next browser launch, Edge resolves the extension ID to the deployed folder and loads the extension.

4.2 GPO Bypass via Extension ID Spoofing

Enterprise deployments of Edge for Business commonly enforce extension allowlists via Group Policy, restricting the browser to a predefined set of extension IDs. A Chromium extension's ID is deterministically derived from the RSA public key declared in its `manifest.json`. Consequently, an attacker who retrieves the public key of an allowlisted extension can generate a malicious extension sharing the same ID.

`stomp` automates this process via its `-spoof` option, which fetches the target extension's manifest from the Microsoft Edge Add-ons store and extracts its public key:

```
python3 stomp.py EXTENSION_A/ \
  --spoof nmhdhpibnnopknkmonacoephklnflpho \
  --prefs-file SecurePreferences \
  --sid "S-1-5-21-XXX-XXX-XXX-XXX" \
  --browser "edge" \
  --target-dir "%LOCALAPPDATA%"
```

The resulting extension is registered under the spoofed ID, satisfying the GPO allowlist while executing attacker-controlled code. This technique is effective regardless of whether the legitimate extension is installed on the target machine.

4.3 Limitations of Baseline Injection

While effective, baseline injection leaves a visible artifact on the filesystem: the extension folder deployed to `%LOCALAPPDATA%` is readable by any process running under the target user. A defender performing a filesystem audit or an EDR solution monitoring extension directories will find the malicious folder and its contents. The following section addresses this limitation through the GAP technique.

5 GAP - Ghost Anchor Persistence

GAP exploits an architectural decoupling between three independent components of Chromium's extension runtime: the `Secure Preferences` file, the Service Worker Registration database, and the ScriptCache. While baseline injection manipulates only the first component, GAP demonstrates that the latter two persist independently of filesystem state, enabling malicious code execution with no corresponding artifact on disk.

5.1 Core Observation

Edge's ScriptCache is indexed by script URL (`chrome-extension://<ID>/background.js`), not by file content. When resolving a service worker at startup, Edge performs the following sequence:

1. Read `Secure Preferences` - identify registered extension IDs and their filesystem paths.
2. Query `Service Worker\Database\` - restore existing Registrations by scope.
3. Verify that the extension folder exists on disk - validate the extension ID.
4. Serve the service worker script from `Service Worker\ScriptCache\` - using the cached entry indexed by script URL.

Critically, **step 4 does not verify the integrity of the cached script against the file currently present on disk**. Edge treats folder existence as sufficient proof of extension validity and unconditionally serves the cached script. This behavior creates an exploitable window: if the cached script originates from a different extension than the one currently pointed to by `Secure Preferences`, the decoupling goes undetected.

5.2 Attack Primitive

GAP requires two extensions sharing the same ID: a malicious extension **A** with a functional background service worker, and a benign extension **B** with identical filenames but empty script content. The infection chain proceeds as follows:

1. **Deploy A** - inject extension A using `inject.bat`. Edge registers A's extension ID in `Secure Preferences` and resolves its folder path.
2. **Trigger Registration** - open and close Edge. This step is mandatory: it causes Edge to register A's service worker in `Service Worker\Database\` and cache its compiled `background.js` in `Service Worker\ScriptCache\`. Omitting this step breaks the persistence chain.
3. **Stomp with B** - execute B's `inject.bat`. `Secure Preferences` is overwritten: the extension ID now resolves to B's benign folder. The Service Worker Registration database and ScriptCache are not affected.
4. **Remove A** - delete A's folder from the filesystem. Only B's benign folder remains on disk. Consequently, no more malicous artifacts remain on the filesystem.

At next browser launch, Edge resolves the extension ID to B's folder (step 1), finds an existing Registration for this scope (step 2), confirms the folder exists (step 3), and loads A's malicious script from ScriptCache (step 4). The Ghost Anchor Persistence is established.

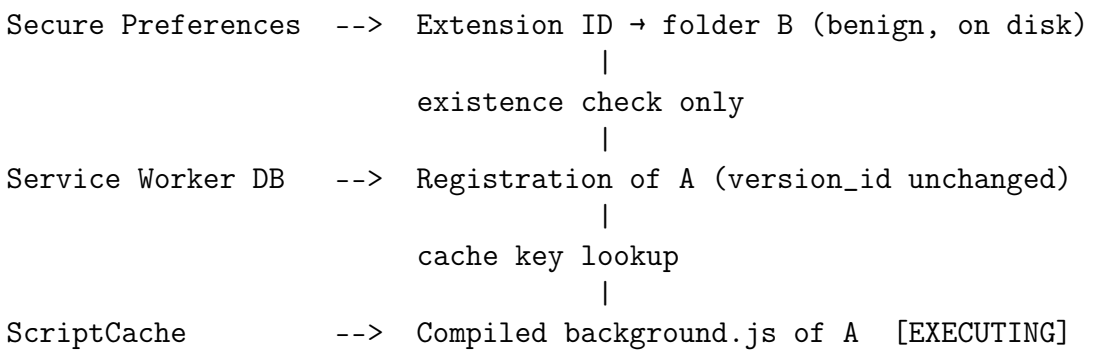

Figure 2: Architectural decoupling enabling GAP

5.3 Why ScriptCache Is Not Invalidated

ScriptCache invalidation in Chromium requires one of three conditions: an HTTP cache-control signal, an explicit `unregister()` call, or a version bump through the official extension update mechanism. None of these are triggered by replacing the extension folder:

- The `chrome-extension://` scheme carries no HTTP headers, cache-control validation does not apply.
- No `unregister()` is called, the Registration persists across the stomp operation.
- The extension ID remains identical between A and B, no version bump is detected by the update mechanism.

As a result, the cache key (`chrome-extension://<ID>/background.js`) remains valid across the stomp, and Edge serves A's cached script indefinitely.

5.4 Forensic Evidence

I provide direct evidence of the decoupling through offline analysis of both persistent stores after GAP execution.

ScriptCache. With Edge terminated, a string search across ScriptCache entries reveals the malicious script cached under the hash of A's script URL, as show with Figure 3:

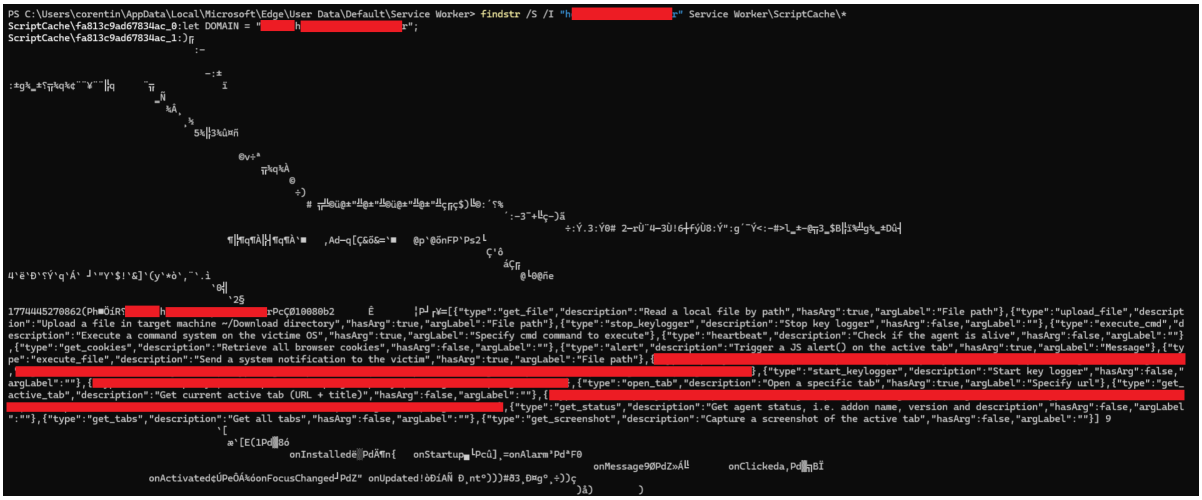

Figure 3: Evidence ScriptCache.

The file `fa813c9ad67834ac_0` contains A's compiled `background.js`, including C2 domain, registered event handlers, and capability declarations; despite A's folder having been removed from disk.

Service Worker Database. String extraction from the Registration database reveals one SHA-256 hash associated with the malicious extension registration of the extension ID, as shown with Figure 4:

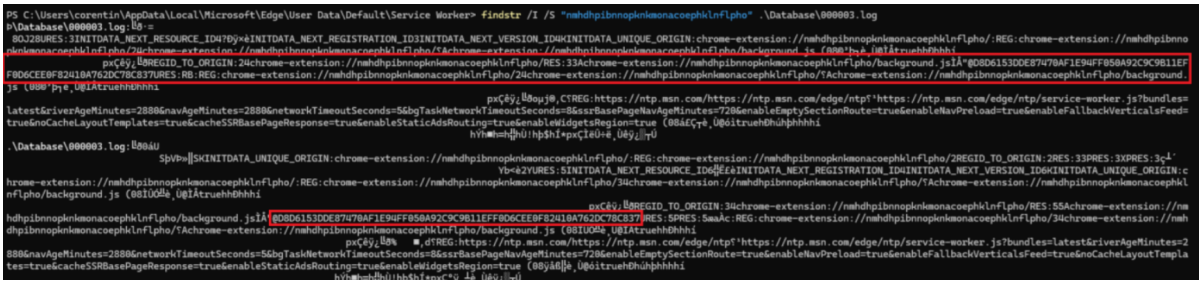

Figure 4: Hash registers in Database Service Worker.

- `E3B0C44...` - SHA-256 of an empty file, corresponding to B's `background.js`
- `D8D6153...` - SHA-256 of A's malicious `background.js`

The Service Worker Registration database retains exclusively `D8D6153...`, the hash of A's malicious background.js, across all successive registration entries, even after the stomp with B. The hash of B's empty script `E3B0C44...` never appears, confirming that Edge never re-registers the service worker after the stomp. The existing Registration is silently reused, loading A's cached script from ScriptCache indefinitely.

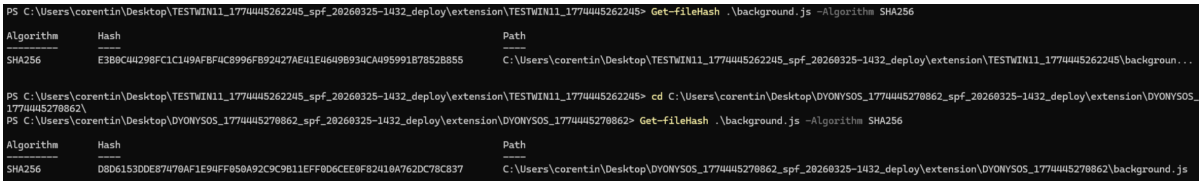

Figure 5: Hash verification.

Hash verification. According to the Figure 5, the match confirms that the hash stored in the Registration database ScriptCache serves A's malicious code.

5.5 Persistence Properties

GAP persistence was validated across the following conditions:

- **Browser restart** - the malicious service worker is restored from ScriptCache at each Edge launch.
- **System reboot** - both the Registration database and ScriptCache survive reboots; GAP persists indefinitely.
- **Browser update** - Edge updates do not invalidate ScriptCache entries for locally registered extensions; GAP survives minor and major browser updates.
- **Filesystem audit** - only B's benign folder is present on disk; no malicious file is recoverable through standard forensic analysis.

6 Implementation

This kind of vulnerability can be really convenient in a redteam mission context or in a kill chain attack. In post exploitation phase, after establishing an initial access on the targeted machine, a stealthy persistent agent can be deployed on the system. In this scenario, the malicious browser extension operates as an agent and can act as a RAT.

6.1 Extension Design

GAP requires two extensions sharing the same ID. Both are generated by `stomp` using the `-spoof` option to derive the correct public key from a target allowlisted extension. Their structure is intentionally minimal:

File	Extension A	Extension B
`manifest.json`	Full permissions	Identical
`background.js`	Malicious payload	Empty
`content.js`	Empty	Empty

Table 2: Extension A and B file structure

A critical requirement is that both extensions declare identical filenames in their manifests. Since ScriptCache entries are indexed by the full script URL (`chrome-extension://<ID>/background.js`), any discrepancy in filename between A and B would cause Edge to attempt a fresh fetch on the next launch, breaking the persistence chain.

6.2 Manifest Configuration

The `manifest.json` must declare the permissions required by the intended payload. A representative configuration for a red team implant is as follows:

```
{
  "manifest_version": 3,
  "name": "<spoofed_name>",
  "version": "1.0.0",
  "key": "<public_key_of_spoofed_extension>",
  "background": {
    "service_worker": "background.js"
  },
  "permissions": [
    "activeTab",
    "cookies",
    "scripting",
    "storage",
    "webRequest"
  ],
  "host_permissions": ["<all_urls>"],
  "content_scripts": [{
    "matches": ["<all_urls>"],
    "js": ["content.js"]
  }]
}
```

The `key` field is populated by `stomp` during generation and determines the extension ID. Both A and B share the same `key` value, ensuring ID consistency across the stomp operation.

6.3 Dynamic Content Script Injection

As noted in Section 5, content scripts are loaded directly from the extension folder at injection time and do not benefit from ScriptCache persistence. B's `content.js` being empty, any static content script payload would be lost after the stomp.

I address this by moving all page-level logic into the background service worker and injecting it dynamically via the `scripting.executeScript()` API:

```
chrome.tabs.onUpdated.addListener((tabId, changeInfo, tab) => {
    if (changeInfo.status === 'complete' && tab.url &&
        !tab.url.startsWith('chrome')) {
        chrome.scripting.executeScript({
            target: { tabId: tabId },
            func: () => {
                // inline payload
                // zero additional filesystem artifact
            }
        });
    }
});
```

This approach achieves functional equivalence with static content script injection while consolidating the entire attack surface within the service worker; the only component that benefits from GAP persistence. The `func` parameter accepts an inline JavaScript function, eliminating any dependency on files present in the extension folder.

6.4 C2 Communication

The background service worker maintains a persistent connection to an attacker-controlled C2 server via a polling loop over `fetch()`. This architecture is well-suited to the service worker execution model, which supports asynchronous network requests natively:

```
const DOMAIN = "c2.attacker.com";
const INTERVAL = 5000;

async function poll() {
    try {
        const res = await fetch(‘https://${DOMAIN}/task‘);
        const task = await res.json();
        await dispatch(task);
    } catch (_) {}
    setTimeout(poll, INTERVAL);
}

chrome.runtime.onInstalled.addListener(poll);
chrome.runtime.onStartup.addListener(poll);
```

6.5 Agent Capabilities

Table 3 summarizes the capabilities implemented in our proof-of-concept agent, all executed exclusively from the background service worker:

Capability	**API**
Cookie exfiltration	`chrome.cookies.getAll()`
Keylogging	`scripting.executeScript()`
Screenshot capture	`tabs.captureVisibleTab()`
Active tab monitoring	`tabs.onActivated`
Credential theft	`scripting.executeScript()`
Command execution	`scripting.executeScript()`
Network proxying	`proxy.settings`

Table 3: Agent capabilities

All capabilities operate within the browser sandbox and require no system-level privileges beyond those declared in the extension manifest. Command execution is achieved by injecting scripts into the active tab's DOM context; this does not constitute a sandbox escape but enables interaction with web application interfaces accessible to the victim user.

7 Evaluation

GAP has been tested several times against MDE (Microsoft Defender for Endpoint), which is a real-time endpoint detection and response. All tests have been successful as not alert was raised by the EDR.

7.1 Experimental Setup

Some experiments were conducted on isolated virtual machines with Microsoft Edge for Business installed in its default configuration. This vulnerability has also been validated on CA-SA machine with enterprise default configuration. No modifications were made to browser security settings prior to testing. Active Directory environments were provisioned with a Windows Server 2022 domain controller enforcing extension allowlist policies via GPO. The target user account held standard privileges in all configurations, no administrative rights were required at any stage of the attack chain.

7.2 Persistence Validation

Table 4 summarizes GAP persistence across all tested environments and conditions.

OS	**Config**	**Post-reboot**	**Post-update**
Windows 11	AD + GPO	✓	✓
Windows 11	Personal + GPO	✓	✓
Windows 11	Personal	✓	✓
Windows 10	Personal + GPO	✓	✓
Windows 10	Personal	✓	✓

Table 4: GAP persistence across tested environments

In all configurations, the malicious service worker was observed active at `edge://serviceworker-internals` with `Installation Status: ACTIVATED` and `Running Status: RUNNING` following folder A deletion, system reboot, and browser update, with no malicious file present on the filesystem, as shown in Figure 6.

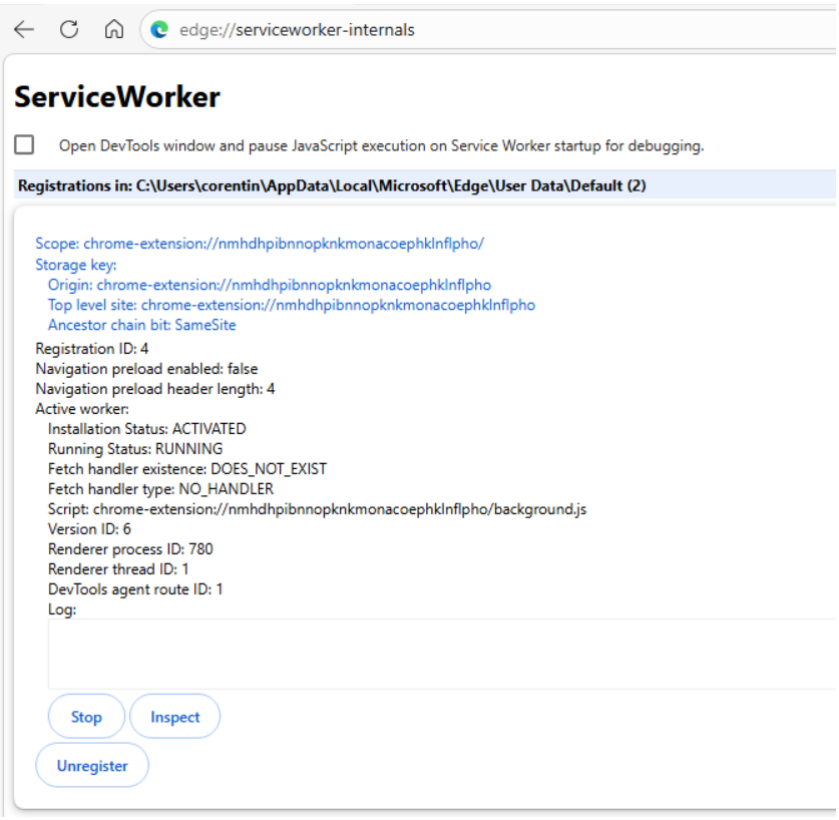

Figure 6: edge://serviceworker-internals.

7.3 AV/EDR Evasion

I evaluated GAP against MDE with real-time filesystem monitoring enabled. Three detection vectors were assessed:

Filesystem scanning. Post-stomp, the only extension-related artifact on disk is B's benign folder containing empty script files and a `manifest.json`. Static analysis of these files yields no malicious indicators. The malicious `background.js` resides exclusively in ScriptCache, stored as a compiled binary blob with no plaintext JavaScript header, reducing its detectability to signature-based tools.

Process monitoring. GAP does not introduce any new process or anomalous process tree. The malicious service worker executes within Edge's existing renderer process infrastructure, indistinguishable from legitimate extension activity at the process level.

Network monitoring. C2 communication is performed via standard HTTPS `fetch()` calls from within the browser process. Traffic is attributed to `msedge.exe` and blends with normal browser network activity. Domain-based detection remains applicable but falls outside the scope of the filesystem and process evasion evaluated here.

7.4 Applicability to Other Chromium Browsers

The vulnerability exploited by GAP resides entirely within Chromium's Service Worker architecture, specifically, the absence of content integrity validation between ScriptCache

entries and their corresponding filesystem files. This behavior is not specific to Microsoft's Edge implementation.

I assessed the structural applicability of GAP to other Chromium-based browsers by examining their storage layouts:

Browser	ScriptCache path	Affected
Microsoft Edge	`%LOCALAPPDATA%\Microsoft \Edge\...`	Confirmed
Google Chrome	`%LOCALAPPDATA%\Google \Chrome\...`	Confirmed
Brave	`%LOCALAPPDATA%\BraveSoftware \...`	Confirmed
Opera	`%APPDATA%\Opera Software \...`	Confirmed

Table 5: Structural applicability to Chromium-based browsers

All listed browsers share Chromium's Service Worker implementation and present an identical directory structure under their respective user data paths.

7.5 Impact

GAP enables an attacker to maintain persistent access to a compromised machine's browser with no recoverable malicious artifact on the filesystem. Table 6 summarizes a concrete capabilities available to an attacker leveraging a GAP-persistent agent.

Capability	Impact
Cookie exfiltration	Session hijacking, account takeover
Keylogging	Credential theft
Screenshot capture	Sensitive data exposure
Network interception	Man-in-the-browser attacks
Credential theft	Access to stored browser passwords
System file reading	Access to file system
Arbitrary execution commands	Possible in sophisticated scenario

Table 6: Attacker capabilities via GAP-persistent agent

From a defensive standpoint, the only filesystem evidence left by GAP is a benign extension folder containing empty script files. The Indicators of Compromise (IoC) are therefore exclusively behavioral:

- Unexpected modification of `Secure Preferences` by a non-browser process
- Active service worker whose script URL does not match any file content on disk
- Extension folder present in `%LOCALAPPDATA%` (or other location) with empty JS files but active browser permissions

Critically, **standard filesystem-based AV/EDR solutions will not detect GAP** as no malicious file is present on disk at any point after the stomp. Detection requires runtime browser state analysis, which is outside the scope of most current endpoint security configurations.

7.6 Responsible Disclosure

A vulnerability report was submitted to the Microsoft Security Response Center (MSRC) on April 2, 2026, under case number **MSRC Case 112111**. Microsoft acknowledged receipt and opened a formal investigation within 24 hours. Following their review, Microsoft assessed the technique as not meeting their bar for a security update, citing the requirement for prior local access as a prerequisite. This assessment may be questionable: while initial access is required, the post-exploitation value of a fileless, EDR-evading persistence primitive operating within a trusted browser process represents a meaningful security risk in enterprise environments.

8 Detection

Detecting GAP requires moving beyond filesystem-based indicators and correlating browser runtime state with on-disk artifacts. I identify three complementary detection strategies.

8.1 Secure Preferences Integrity Monitoring

The first observable event in the GAP infection chain is the modification of the `Secure Preferences` file. EDR solutions or SIEM pipelines ingesting Sysmon telemetry can alert on unexpected writes to this file:

```
Event ID  : 11 (FileCreate)
TargetFilename: *\Microsoft\Edge\User Data\Secure Preferences
```

Legitimate modifications to `Secure Preferences` occur during browser updates and official extension installations. Suspicious writes are characterized by:

- The writing process being `cmd.exe`, `powershell.exe`, or any non-Edge process
- Modification occurring while Edge is not running
- Two successive writes within a short time window - corresponding to the A and B injection steps

8.2 Service Worker Runtime Auditing

The most direct detection vector is a runtime comparison between active service workers and their corresponding filesystem artifacts. For each Registration present in `edge://serviceworker-internals`, a defender can extract the `script_url` field and verify that the file at the corresponding path on disk matches the cached entry.
A hash mismatch between the Registration database entry and the current filesystem file is a strong indicator of a GAP-type attack. Under normal operation, these values should always be consistent.

8.3 ScriptCache Detection

A complementary approach consists of identifying ScriptCache entries whose corresponding extension folder no longer contains a matching script. The `index` file in `Service Worker\ScriptCache\` maps script URLs to their cache hash. An entry referencing a `chrome-extension://` URL whose corresponding file is absent or empty on disk is an anomalous condition that warrants investigation.

8.4 Detection Summary

Method	Coverage	Complexity	Reliability
Secure Preferences monitoring	Early	Low	Medium
Runtime hash comparison	Post-infection	Medium	High
ScriptCache orphan detection	Post-infection	High	High

Table 7: Detection methods comparison

I recommend deploying all three strategies in combination. `Secure Preferences` monitoring provides early warning during the injection phase, while runtime hash comparison and ScriptCache auditing cover the post-infection persistent state.

9 Conclusion

This paper presented GAP (Ghost Anchor Persistence), a novel fileless persistence technique affecting Microsoft Edge for Business and, by extension, all Chromium-based browsers. GAP exploits an architectural decoupling between three independent components of Chromium's extension runtime, the `Secure Preferences` file, the Service Worker Registration database, and the ScriptCache, to achieve persistent arbitrary JavaScript execution within the browser with no malicious artifact remaining on the filesystem.

We demonstrated that ScriptCache invalidation is never triggered by replacing an extension's folder on disk, as the cache is indexed by script URL rather than content. This fundamental design gap allows an attacker with an initial foothold on a Windows machine to deploy a malicious service worker, replace its folder with a benign decoy, and maintain persistent C2 access across reboots and browser updates, while leaving only empty script files as forensic evidence.

Our evaluation confirmed GAP's effectiveness across four distinct Windows configurations, including Active Directory environments with GPO-enforced extension allowlists. We provided direct forensic evidence of the decoupling through offline analysis of ScriptCache and the Service Worker Registration database, demonstrating a measurable hash mismatch between the on-disk state and the active browser runtime. We further showed that the limitation of GAP to background service workers, content scripts being loaded from disk, is fully addressed through dynamic injection via `scripting.executeScript()`, consolidating the entire attack surface within a single cached component.

A responsible disclosure was submitted to Microsoft on April 2, 2026 (MSRC Case 112111). Microsoft assessed the technique as not meeting their bar for a security update. We maintain that the post-exploitation value of a fileless, EDR-evading persistence primitive operating within a trusted browser process represents a meaningful and underappreciated risk in enterprise environments. A valuable detection method would be to implement content integrity validation between ScriptCache entries and their corresponding filesystem artifacts, a targeted fix that would eliminate this class of attack.

References

[1] Stomp (2026). *Tool enabling injection and GPO bypass in Chromium-based browsers.* https://github.com/Fir3n0x/stomp

[2] Synacktiv, *L'extension fantôme - Infiltrer Chrome par des voies inexplorées*, 2024. https://www.synacktiv.com/publications/lextension-fantome-infiltrer-chrome-par-des-voies-inexplorees

[3] Secure Preferences paper Chromium-based browser security (2020). *HMAC and "Secure Preferences": Revisiting Chromium-based Browsers Security.* Disponible sur : https://issues.chromium.org/issues/40149897

[4] Tripwire (2017). *Hackers Hijack Popular Chrome Extension, Inject Code Into Web Developers' Browsers.* Disponible sur : https://www.tripwire.com/state-of-security/hackers-hijack-popular-chrome-extension-inject-code-web-developers-browsers

[5] GeekWire (2014). *Google disables two popular Chrome extensions following adware complaints.* Disponible sur : https://www.geekwire.com/2014/google-disables-two-popular-chrome-extensions-following-adware-complaints/

[6] Aikido Security (2026). *GlassWorm: Chrome Extension RAT.* Disponible sur : https://www.aikido.dev/blog/glassworm-chrome-extension-rat